

SQL Notes — Aggregate Functions & NULL Values

Exam Revision Notes • DBMS Core Concepts

MASTER SHEET

1 What is a NULL Value?

CRITICAL RULE: NULL ≠ 0

- 0 is a real numeric value.
- NULL means the value is **unknown, missing, or not available**.
- NULL does *not* mean empty string "" or zero.

Example Table:

ID	Name	Salary	Status / Meaning
1	Ravi	10000	Known salary
2	Aman	NULL	Salary is <i>unknown/not available</i> (not zero)
3	Nitin	30000	Known salary

2 Aggregate Functions Overview

Aggregate functions perform calculations on a set of rows and return a single scalar result.

Function	Purpose	NULL Behavior
MAX()	Finds the highest value	Ignores NULL
MIN()	Finds the lowest value	Ignores NULL
COUNT()	Counts values or rows	COUNT(*) includes NULL; COUNT(col) ignores NULL
SUM()	Calculates the total sum	Ignores NULL
AVG()	Calculates the average	Ignores NULL

🧠 Memory Trick for Aggregate Functions:

M - M - C - S - A → Max, Min, Count, Sum, Average

3 Detailed Breakdown: MAX(), MIN(), and COUNT()

♦ MAX() & MIN()

Sample salaries: 10000, 20000, 30000, 50000, NULL

```
SELECT MAX(salary) FROM emp; -- Output: 50000
SELECT MIN(salary) FROM emp; -- Output: 10000
```

Both functions ignore NULL values entirely.

♦ COUNT(*) vs COUNT(column)

Sample salaries: 10000, 20000, 30000, 30000, 50000, NULL (6 rows)

```
SELECT COUNT(*) FROM emp; -- Output: 6
SELECT COUNT(salary) FROM emp; -- Output: 5
```

COUNT(*) counts rows. COUNT(column) ignores NULL.

4 COUNT(DISTINCT column)

DISTINCT removes duplicate values before the count is performed.

Dataset: 10000, 20000, 30000, 30000, 50000, NULL

```
SELECT COUNT(DISTINCT salary) FROM emp;
```

Unique Non-NULL Values: 10000, 20000, 30000, 50000 → Output: 4

5 SUM() and AVG() Functions

Dataset: 10000, 20000, 30000, 30000, 50000, NULL

♦ SUM() Function

```
SELECT SUM(salary) FROM emp;
```

Calculation: $10000 + 20000 + 30000 + 30000 + 50000 = 140000$

SUM with DISTINCT:

```
SELECT SUM(DISTINCT salary) FROM emp;
```

Calculation: $10000 + 20000 + 30000 + 50000 = 110000$ (Duplicate 30000 counted once).

♦ AVG() Function

```
SELECT AVG(salary) FROM emp;
```

Formula: $AVG = SUM / COUNT(\text{non-NULL values})$

Calculation: $140000 / 5 = 28000$

Note: The denominator is 5 (non-NULL rows), NOT 6 (total rows)!

6 Aggregate Functions + NULL Evaluation ★★

Consider Table T1 with 5 total rows:

Row	Column A	Column B
1	1	NULL
2	2	NULL
3	3	NULL
4	4	NULL
5	NULL	NULL

Query	Output	Reason / Calculation
COUNT(*)	5	Counts all 5 rows
COUNT(A)	4	Ignores 1 NULL value
COUNT(B)	0	All values in B are NULL
SUM(A)	10	$1 + 2 + 3 + 4 = 10$
AVG(A)	2.5	$10 / 4 = 2.5$
MIN(A) / MAX(A)	1 / 4	Ignores NULL

🔥 Most Important NULL Rule for Exams:

For a column containing **ONLY NULL values** (like Column B above):

- COUNT(B) → 0
- SUM(B), AVG(B), MIN(B), MAX(B) → NULL
- COUNT(*) → Total number of rows in the table.

7 Department-wise Aggregations & GROUP BY

GROUP BY splits rows into groups, and aggregate functions compute results for each group.

```
SELECT department, MAX(salary)
FROM emp
GROUP BY department;
```

Sample Grouping Result

Department	MAX(Salary)
HR	30000
IT	50000
Marketing	40000

Common GROUP BY Patterns

```
-- Department row count
SELECT dept, COUNT(*) FROM emp GROUP BY dept;

-- Department salary spending
SELECT dept, SUM(salary) FROM emp GROUP BY dept;

-- Department average salary
SELECT dept, AVG(salary) FROM emp GROUP BY dept;
```

8 SQL Query Execution Order ★★

Database engines evaluate SQL clauses logically in a specific sequential order:

1	2	3	4	5	6	7	8
FROM	JOIN	WHERE	GROUP BY	HAVING	SELECT	DISTINCT	ORDER BY
9 LIMIT							

🧠 Memory Trick:

F - J - W - G - H - S - D - O - L

(From → Join → Where → Group BY → Having → Select → Distinct → Order BY → Limit)

Execution Example

```
SELECT department, COUNT(department)
FROM employee
WHERE city = 'Delhi'
GROUP BY department
HAVING COUNT(department) < 5
ORDER BY COUNT(department) DESC
LIMIT 1;
```

9 Subquery Operators: IN, NOT IN, EXISTS, NOT EXISTS

♦ IN vs NOT IN

Compares a value against a list or subquery result.

```
-- Address is Delhi, Pune, or Chandigarh
SELECT * FROM emp
WHERE address IN ('Delhi', 'Pune', 'Chandigarh');

-- Address is NOT any of those 3
SELECT * FROM emp
WHERE address NOT IN ('Delhi', 'Pune', 'Chandigarh');
```

♦ EXISTS vs NOT EXISTS

Evaluates boolean existence in correlated subqueries.

```
-- Employees working on at least 1 project
SELECT * FROM emp e
WHERE EXISTS (
    SELECT 1 FROM project p WHERE e.eid = p.eid
);

-- Employees NOT working on any project
SELECT * FROM emp e
WHERE NOT EXISTS (
    SELECT 1 FROM project p WHERE e.eid = p.eid
);
```

Summary: Subqueries

- **IN / NOT IN** → Compare single value against multiple values.
- **EXISTS** → TRUE if matching row exists in subquery.
- **NOT EXISTS** → TRUE if matching row does NOT exist in subquery.

10 LIKE Operator (Pattern Matching) ★★

Used for string pattern matching using wildcards: `%` (0 or more chars) and `_` (exactly 1 char).

Pattern	Meaning	Example Matches
'A%'	Starts with 'A'	Arun, Aman, Ajeet
'%N'	Ends with 'N'	Varun, Aman
'%ee%'	Contains 'ee' anywhere	Sangeeta, Geeta
'_A%'	2nd character is 'A'	Varun, Raj
'_A____'	Exactly 5 characters, 2nd is 'A'	Varun, Raman

11 SQL SEQUENCE

A database object used to auto-generate unique numeric values (commonly for primary keys).

```
CREATE SEQUENCE emp_seq
START WITH 100
INCREMENT BY 1;
```

Fetch Next Value:

```
SELECT emp_seq.NEXTVAL FROM dual;
-- Returns: 100, 101, 102, 103...
```

Option	Meaning
START WITH	Initial value
INCREMENT BY	Step difference
MAXVALUE / MINVALUE	Upper / lower bounds
CYCLE / NOCYCLE	Restart on reaching limit

Exam Quick Revision

Aggregate Functions & NULL

- `MAX()` → Highest value
- `MIN()` → Lowest value
- `COUNT(*)` → Counts ALL rows (including NULLs)
- `COUNT(column)` → Counts non-NULL values
- `SUM()` / `AVG()` → Ignore NULLs

Pattern Matching & Subqueries

- `%` → Any number of characters
- `_` → Exactly ONE character
- `IN` → Belongs to list
- `EXISTS` → Subquery returns ≥ 1 row

🔥 5 Lines You Must Remember for Exams:

- NULL is not 0.** It represents unknown or missing data.
- COUNT(*)** counts rows, including rows containing NULL.
- COUNT(column)** and all other aggregate functions ignore NULL values.
- %** means any number of characters; **_** means exactly one character.
- GROUP BY** creates groups; aggregate functions calculate results per group.